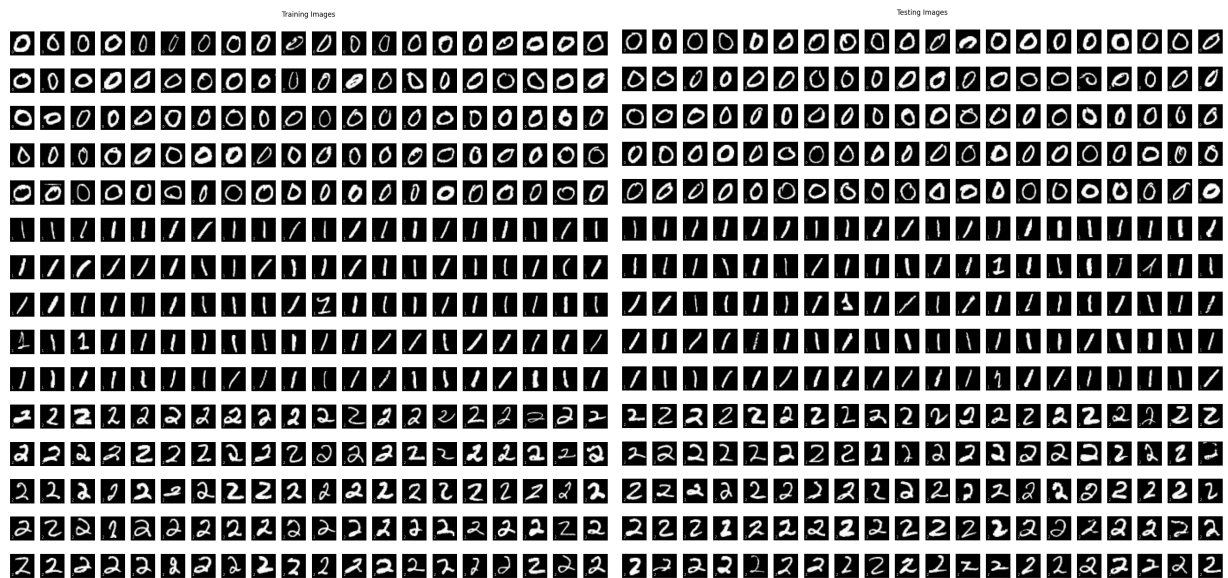


SML Assignment 2

1. Dataset used

- Digits used: 0, 1, 2
- 100 samples per class (train) = 300 total
- 100 samples per class (test) = 300 total
- Original image shape: 28×28
- Images reshaped to 784-dimensional column vectors
- Training data matrix shape: (784, 300)
- Test data matrix shape: (784, 300)
- Label vector shape (train): (300,)
- Label vector shape (test): (300,)
- Pixel values normalized to range [0, 1]



2. Implementation

I. Principal Component Analysis (PCA)

Since our data has 28×28 dimensions, we need to reduce them to the most important dimensions. Thus PCA is used to reduce the dimension of the data while preserving maximum variance of the data.

(i) Build Data Matrix & Normalise

```
X_train = X_train.reshape(X_train.shape[0], -1).T    # (784, 300)
X_train = X_train / 255.0
```

(ii) Compute Mean & Centre

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad X_c = X - \mu$$

```
mu = (1/N) * np.sum(X, axis=1, keepdims=True)
Xc = X - mu
```

(iii) Compute Covariance Matrix

$$S = \frac{X_c X_c^T}{N - 1}$$

```
S = (Xc @ Xc.T) / (N - 1)
```

(iv) Eigendecomposition & Sort Descending by Eigenvalues

We seek the direction u that maximises the projected variance, subject to $\|u\| = 1$:

$$\max_u u^T S u \quad \text{subject to} \quad u^T u = 1$$

Using Lagrange multiplier:

$$\frac{\partial}{\partial u} [u^T S u - \lambda(u^T u - 1)] = 0 \Rightarrow S u = \lambda u$$

This is the eigenvalue equation. The maximum variance is achieved by the eigenvector with the largest eigenvalue.

```
eigvals, eigvecs = np.linalg.eigh(S)
pairs = sorted(zip(eigvals, eigvecs.T), reverse=True, key=lambda x: x[0])
```

(v) Select p Components by Variance Threshold

$$p = \min \left\{ k : \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i} \geq \tau \right\} \quad U_p = [u_1, \dots, u_p]$$

```
cumsum = 0
for i, val in enumerate(eigvals):
    cumsum += val
    if (cumsum / total) >= variance:
        p = i + 1; break
Up = eigvecs_sorted[:, :p]
```

(vi) Project Data

$$Y = U_p^T X_c \quad y_{\text{test}} = U_p^T (x_{\text{test}} - \mu)$$

```
Y      = Up.T @ Xc
y_test = Up.T @ (x_test - mu)
```

(vii) Reconstruct & Compute MSE

$$\hat{X} = U_p Y + \mu \quad \text{MSE} = \frac{1}{dN} \|X - \hat{X}\|_F^2$$

```
X_hat = Up @ Y + mu
mse    = np.mean(np.square(X_hat - X_train))
```

II. Fisher's Discriminant Analysis (FDA)

FDA finds directions onto which the data can be projected such that the class separation is maximized while minimizing the scatter within the class. Thus it helps finding better decision boundary for classification.

(i) Compute Global and Class Means

$$\mu = \frac{1}{N} \sum x_i \quad \mu_c = \frac{1}{N_c} \sum_{x_i \in C_c} x_i$$

```
mu    = (1/N) * np.sum(X, axis=1, keepdims=True)
mu_c  = (1/Nc) * np.sum(X_c, axis=1, keepdims=True)
```

(ii) Between-Class Scatter Matrix S_B

$$S_B = \sum_c N_c (\mu_c - \mu)(\mu_c - \mu)^T$$

```
SB = np.zeros((d, d))
for c in classes:
    SB += Nc * (mu_c - mu) @ (mu_c - mu).T
```

(iii) Within-Class Scatter Matrix S_W

$$S_W = \sum_c \sum_{x_i \in C_c} (x_i - \mu_c)(x_i - \mu_c)^T$$

```
SW = np.zeros((d, d))
for c in classes:
    diff = X_c - mu_c
    SW += diff @ diff.T
```

(iv) Solve Generalised Eigenvalue Problem

We seek the direction w that maximises the ratio of between-class to within-class scatter, subject to $w^T S_W w = 1$:

$$\max_w w^T S_B w \quad \text{subject to} \quad w^T S_W w = 1$$

Using Lagrange multiplier:

$$\frac{\partial}{\partial w} [w^T S_B w - \lambda(w^T S_W w - 1)] = 0 \Rightarrow S_B w = \lambda S_W w \Rightarrow S_W^\dagger S_B w = \lambda w$$

This is the eigenvalue equation. We take top $C - 1 = 2$ eigenvectors.

$$W = [w_1, w_2]$$

```
eigvals, eigvecs = np.linalg.eigh(np.linalg.pinv(SW) @ SB)
idx = np.argsort(eigvals)[::-1]
W = np.real(eigvecs[:, idx[:len(classes)-1]]))
```

(v) Project Data

$$Z = W^T X \in \mathbb{R}^{2 \times N}$$

```
Xtr_fda = W.T @ X_train
Xte_fda = W.T @ X_test
```

III. Linear Discriminant Analysis (LDA)

LDA classifies samples with the assumption that data is Gaussian and all classes follow the same covariance. It computes a linear discriminant value for a sample corresponding to each class and the class having the highest value indicates the maximum likelihood of belonging to that class.

(i) Estimate Class Means

$$\mu_c = \frac{1}{N_c} \sum_{x_i \in C_c} x_i$$

```
mu = (1/Nc) * np.sum(Xc, axis=1)
```

(ii) Estimate Per-Class Covariance

$$\Sigma_c = \frac{1}{N_c} \sum_{x_i \in C_c} (x_i - \mu_c)(x_i - \mu_c)^T$$

```
centered = Xc - mu.reshape(-1, 1)
cov_c = (1/Nc) * (centered @ centered.T)
```

(iii) Estimate Shared Covariance & Regularize

$$\Sigma = \frac{1}{C} \sum_c \Sigma_c \quad \Sigma_{\text{reg}} = \Sigma + \lambda I, \quad \lambda = 10^{-3}$$

```
shared_cov = (1/len(classes)) * sum(covs.values())
shared_cov += 1e-3 * np.eye(d)
cov_inv = np.linalg.inv(shared_cov)
```

(iv) Linear Discriminant Score

$$g_c(x) = (\Sigma_{\text{reg}}^{-1} \mu_c)^T x - \frac{1}{2} \mu_c^T \Sigma_{\text{reg}}^{-1} \mu_c$$

```
w = cov_inv @ mu_c
g_c = w.T @ X - 0.5 * mu_c.T @ cov_inv @ mu_c
```

(v) Predict

$$\hat{y} = \arg \max_c g_c(x)$$

```
y_pred = np.argmax(np.vstack(lda_scores), axis=0)
```

IV. Quadratic Discriminant Analysis (QDA)

QDA classifies samples with the assumption that data is Gaussian, however each class can follow its own covariance. It computes a quadratic discriminant value for a sample corresponding to each class and the class having the highest value indicates the maximum likelihood of belonging to that class.

(i) Estimate Class Means

$$\mu_c = \frac{1}{N_c} \sum_{x_i \in C_c} x_i$$

```
mu = (1/Nc) * np.sum(Xc, axis=1)
```

(ii) Estimate Per-Class Covariance & Regularize

$$\Sigma_{c,\text{reg}} = \Sigma_c + \lambda I, \quad \lambda = 10^{-3}$$

```
centered = Xc - mu.reshape(-1, 1)
cov_c     = (1/Nc) * (centered @ centered.T)
cov_c     += 1e-3 * np.eye(d)
```

(iii) Compute Log-Determinant

$$\frac{1}{2} \log |\Sigma_{c,\text{reg}}|$$

```
sign, log_det = np.linalg.slogdet(cov_c)
```

(iv) Quadratic Discriminant Score

$$g_c(x) = -\frac{1}{2}(x - \mu_c)^T \Sigma_{c,\text{reg}}^{-1}(x - \mu_c) - \frac{1}{2} \log |\Sigma_{c,\text{reg}}|$$

```
cov_inv = np.linalg.inv(cov_c)
Xc_     = X - mu_c.reshape(-1, 1)
quad_term = np.diag(Xc_.T @ cov_inv @ Xc_)
g_c      = -0.5 * quad_term - 0.5 * log_det
```

(v) Predict

$$\hat{y} = \arg \max_c g_c(x)$$

```
y_pred = np.argmax(np.vstack(qda_scores), axis=0)
```

3. Experiment 0: LDA & QDA

Accuracy

Method	Train	Test
LDA	1.00	0.903333
QDA	1.00	0.990000

Observation

- Both methods achieve perfect training accuracy
- QDA performs better on the test set than LDA because of class-specific covariance

4. Experiment 1: FDA + LDA & QDA

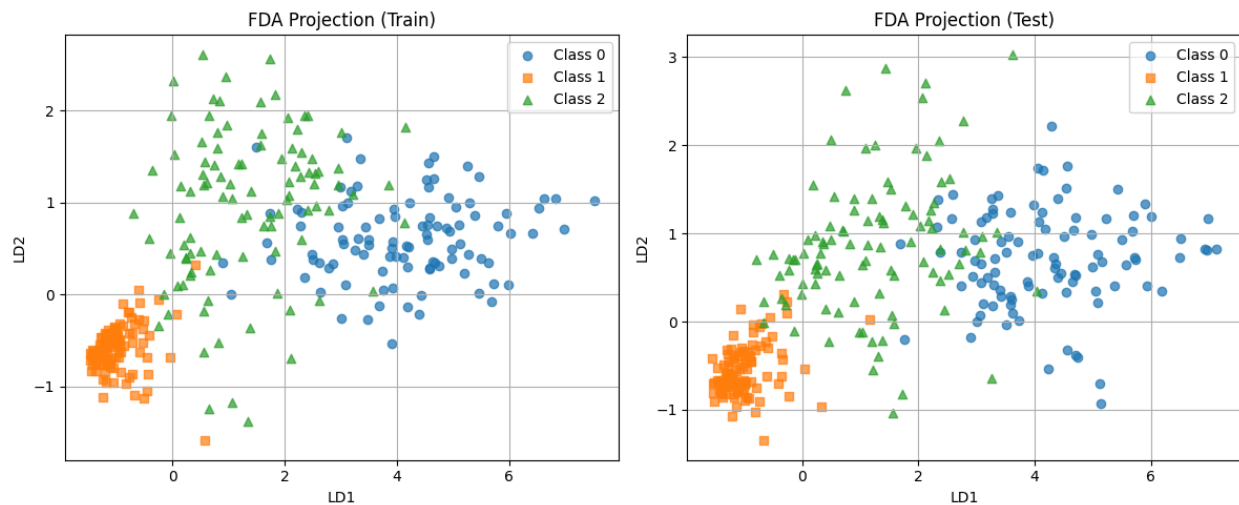
Accuracy

Method	Train	Test
FDA + LDA	0.890000	0.910000
FDA + QDA	0.916667	0.913333

Observation

- FDA projection reduces dimensionality to 2 (C-1) dimensions leading to lower accuracy
- Train and test accuracy are similar (around 0.91)
- FDA focuses on class separation but some overlap still remains

FDA Visualization



Analysis:

- FDA projection shows reasonable separation of classes in 2 dimension
- Some overlap between classes is visible in both training and test projections

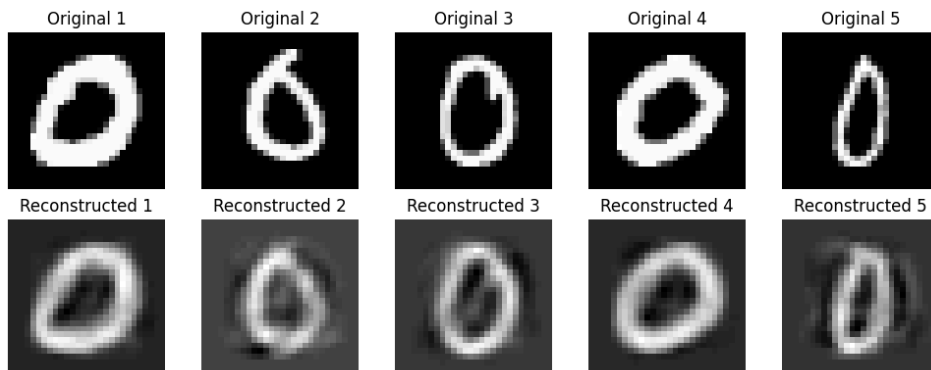
5. Experiment 2: PCA (75% variance) + LDA & QDA

Accuracy

Method	Train	Test
PCA + LDA	0.970000	0.943333
PCA + QDA	0.993333	0.973333

Reconstruction

MSE: 0.0171016623879374



Observation:

- Small gap between train and test accuracy indicates good generalization
- Low reconstruction error (MSE) shows PCA retains overall structure
- QDA outperforms LDA

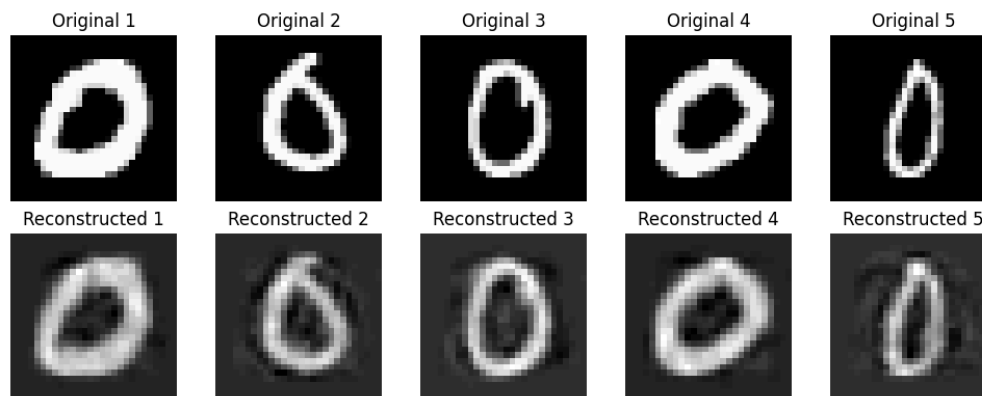
6. Experiment 3: PCA (90% variance) + LDA & QDA

Accuracy

Method	Train	Test
PCA + LDA	0.996667	0.960000
PCA + QDA	1.000000	0.976667

Reconstruction

MSE: 0.006740694449539526



Observation:

- Highest test accuracy (0.96) among PCA-LDA experiments
- MSE further decreased when variance increased to 90%
- Increasing variance improved classification performance as it preserves more discriminative information for the LDA

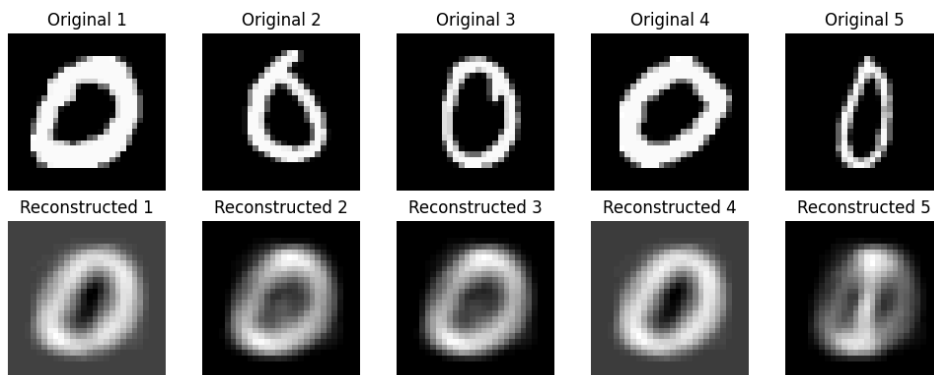
7. Experiment 4: PCA (2 components) + LDA & QDA

Accuracy

Method	Train	Test
PCA + LDA	0.926667	0.923333
PCA + QDA	0.936667	0.940000

Reconstruction

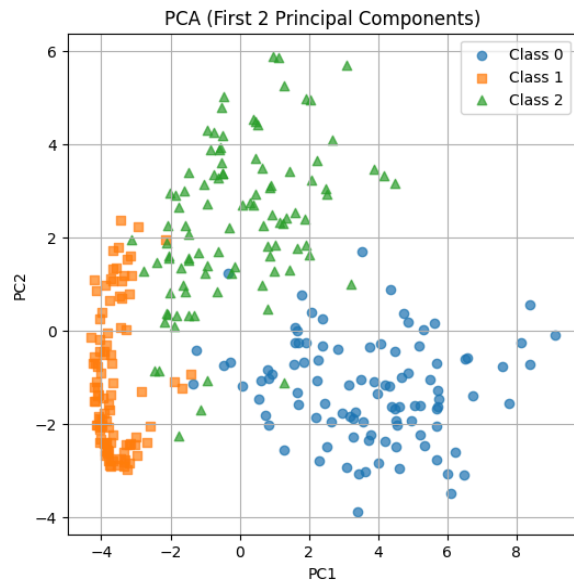
MSE: 0.04764324669751367



Observation:

- Achieves moderate test accuracy= 0.92
- Reducing the data to only two principal components loses some discriminative information (increase in MSE)

PCA 2D Visualization



Analysis: The PCA projection shows only partial class separation, as some overlap between classes remains. This is because PCA maximizes variance rather than class separability.

8. Comparison of results

Method of classification	Train Accuracy	Test Accuracy
LDA	1.000000	0.903333
QDA	1.000000	0.990000
FDA + LDA	0.890000	0.910000
FDA + QDA	0.916667	0.913333
PCA (75% variance) + LDA	0.970000	0.943333
PCA (75% variance) + QDA	0.993333	0.973333
PCA (90% variance) + LDA	0.996667	0.960000
PCA (90% variance) + QDA	1.000000	0.976667
PCA (2 components) + LDA	0.926667	0.923333
PCA (2 components) + QDA	0.936667	0.940000

Note: Since the dataset is perfectly balanced, accuracy is a good measure of evaluating the model's performance. Hence Precision, Recall and F1 score were not computed.

9. Conclusion

- QDA gives the **highest test accuracy** (0.99) among all the tested methods due to use of **class-specific covariance**.
- Applying **PCA before classification improves** generalization. For example, PCA (90%) + LDA gives 0.96 test accuracy compared to using LDA alone which gives 0.90 test accuracy.
- **Increasing retained variance** from 75% to 90% **improves classification accuracy** as retaining more variance preserves more discriminative information.
- Using the first **two principal components reduces accuracy** which shows loss of information.
- Using only **2 components** reduces classification accuracy indicating **loss of important information**.
- **FDA** combined with LDA or QDA performed the **worst** because FDA projects the data to only $C-1=2$ **dimensions** for $C=3$ classes which loses the discriminatory information necessary for classification.
- **PCA (90%) + QDA** achieved the best PCA performance with test accuracy= 0.97.